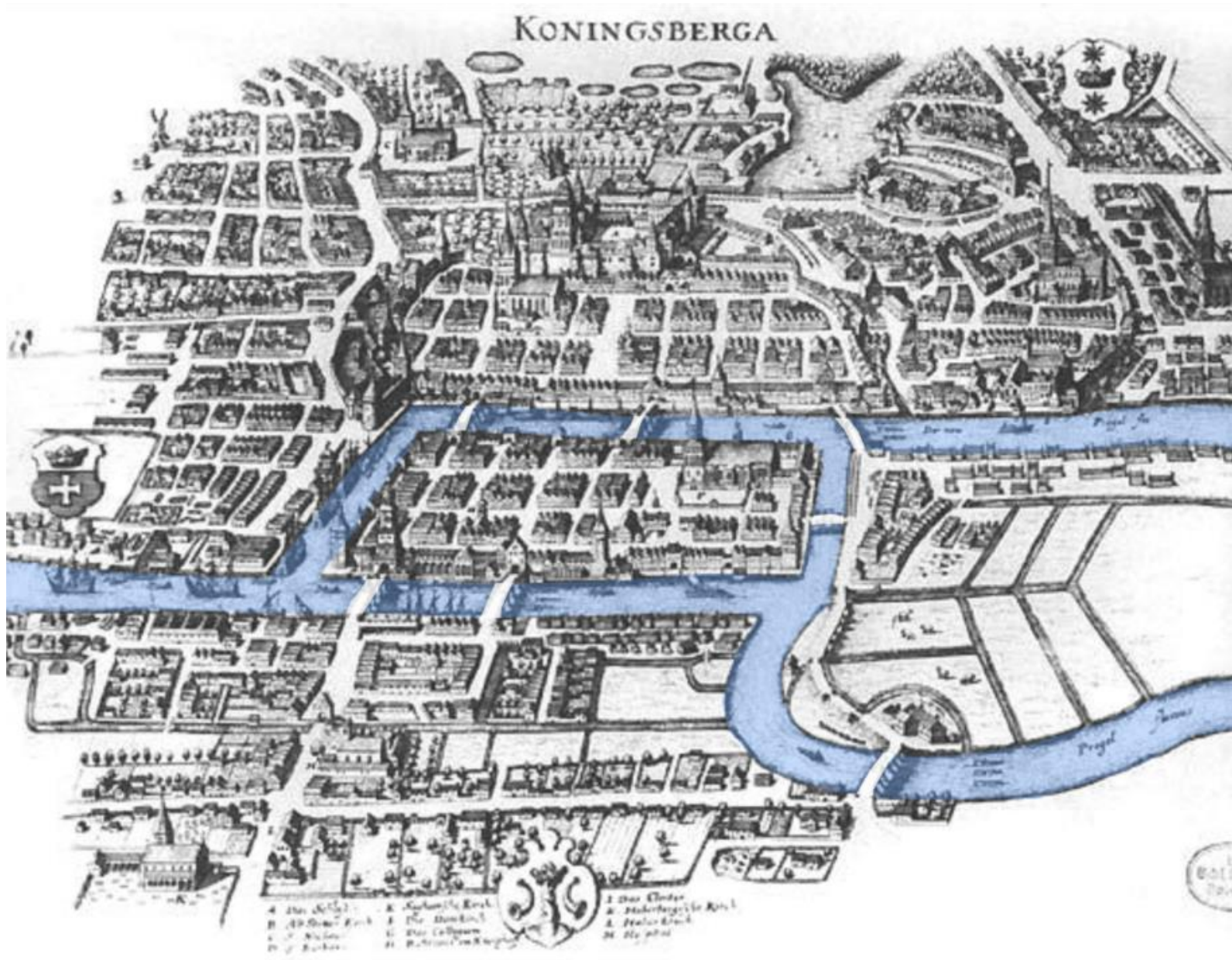




GRAPH DB

Neo4j

1

The graph
model

2

Native/non native

3

Design pattern

4

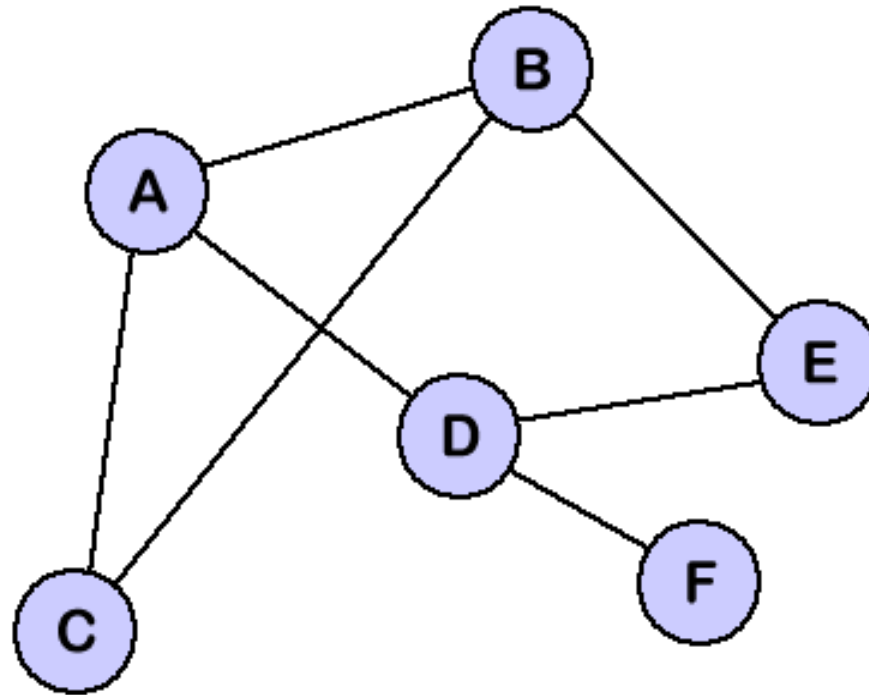
Neo4j

5

Query language

GRAPH

■ $G=(V,E)$



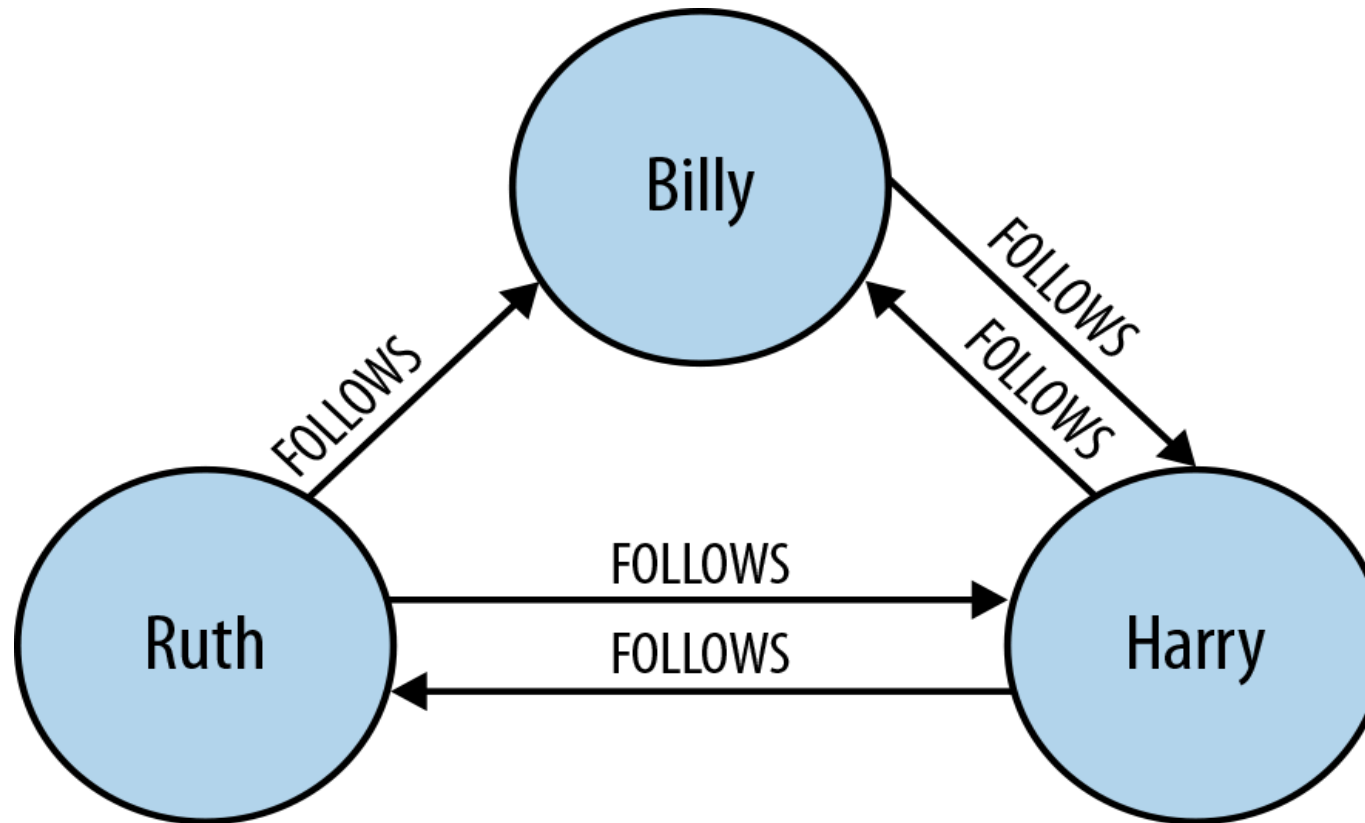
GRAPH

- It is a collection of nodes and edges
- A graph is a set of nodes and their relations
- In a graph
 - First class citizen concepts are modelled as nodes
 - The way in which nodes are semantically connected is modelled with edges

APPLICATION

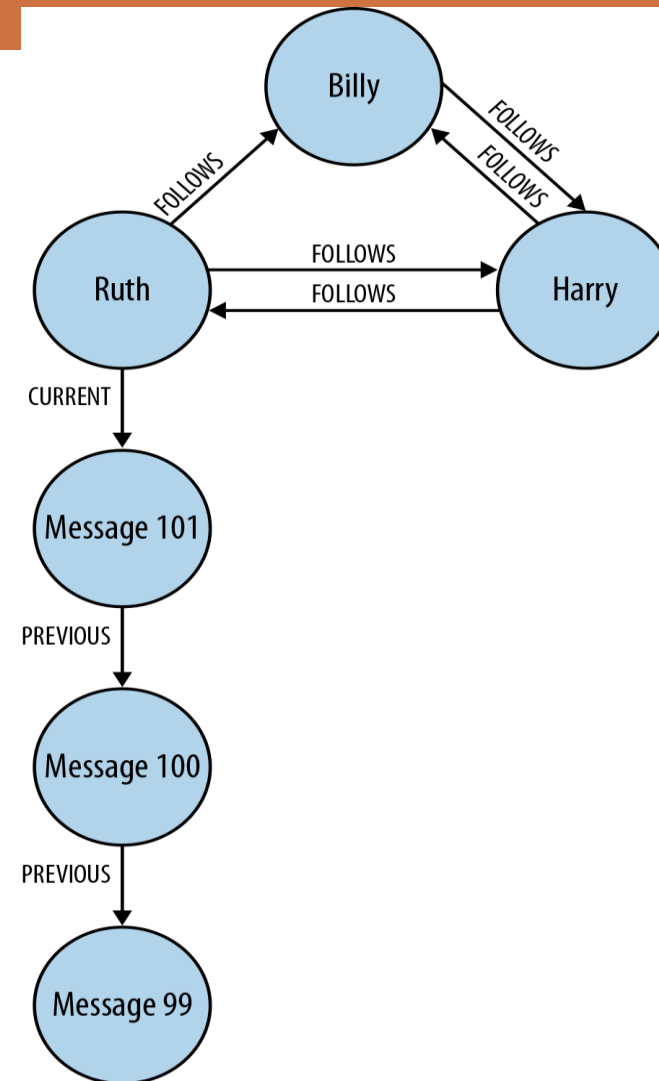
- Social
- Recommendations
- Geo
- Logistics Networks : for package routing, finding shortest Path
- Financial Transaction Graphs : for fraud detection
- Master Data Management
- Bioinformatics
- Authorization and Access Control : *Adobe Creative Cloud, Telenor*

EXAMPLE – TWITTER'S DATA



EXAMPLE – TWITTER'S DATA

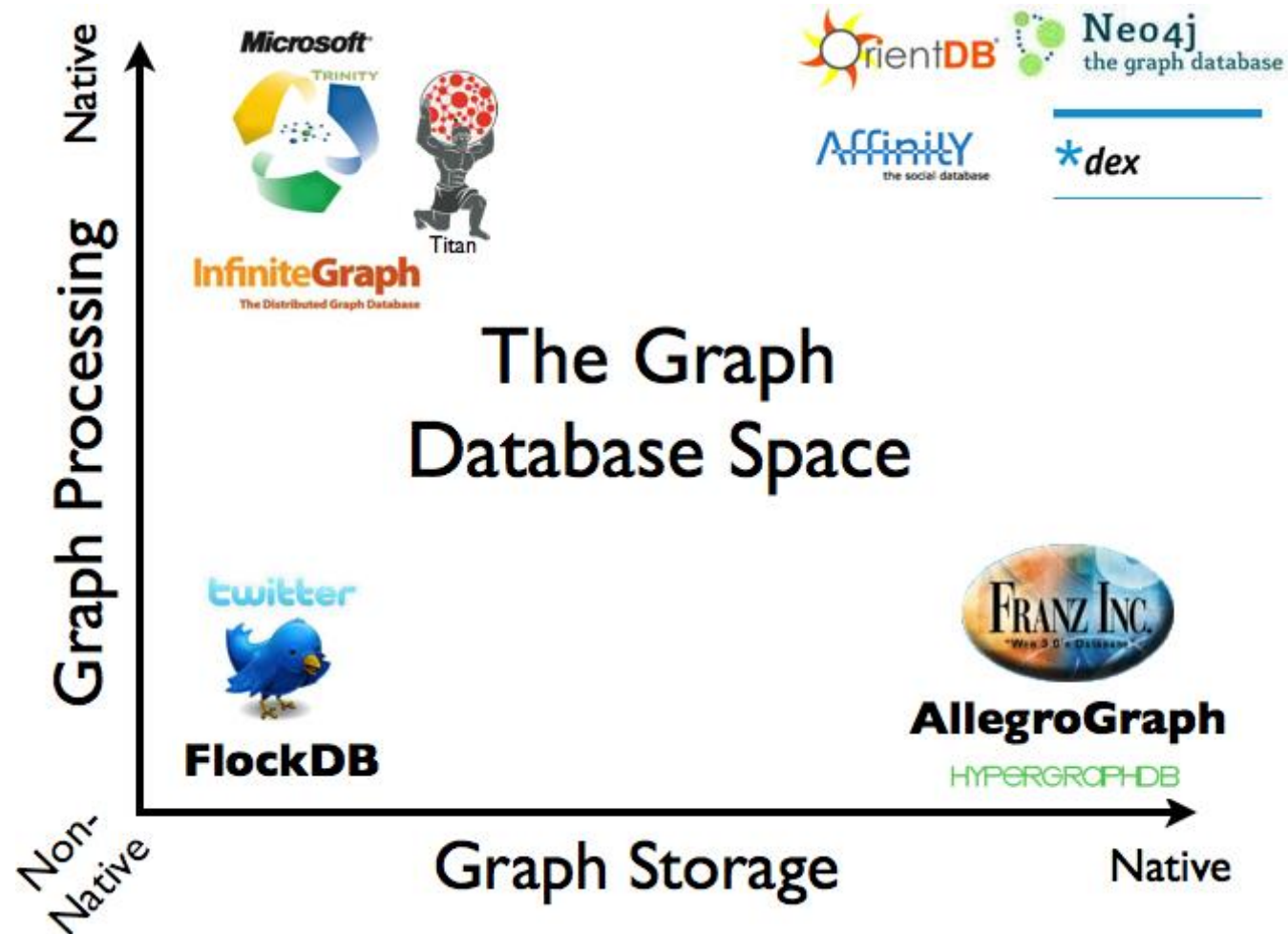
- Data and schema are described in the same way



GRAPH DATABASE PROPERTIES

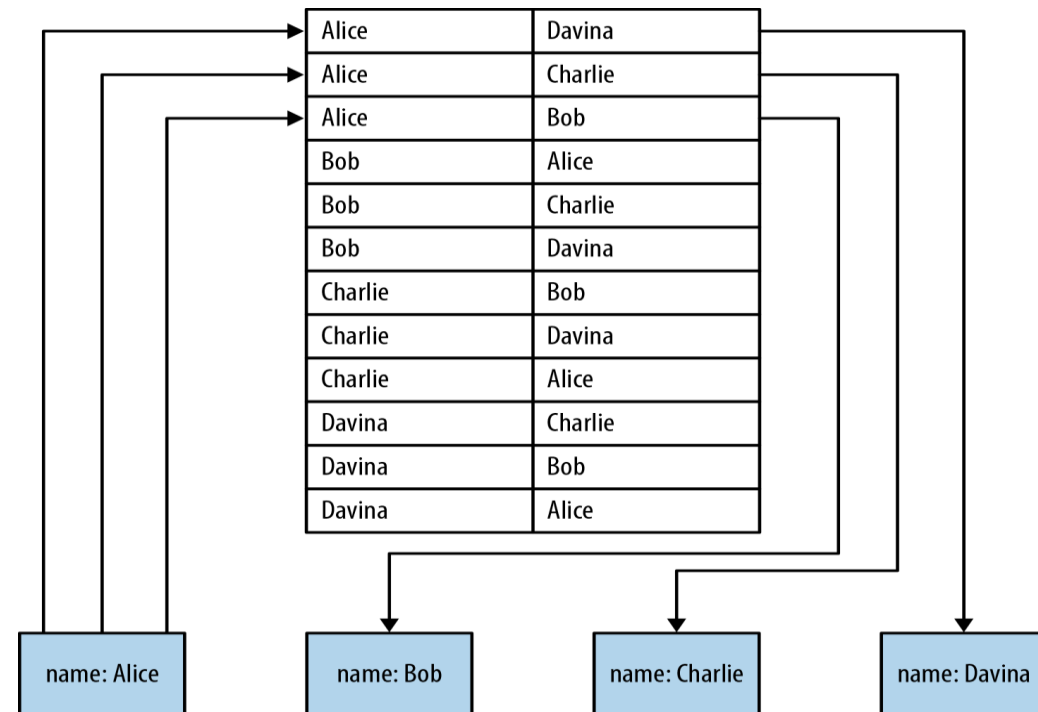
- Storage : Native / Non-Native
- Processing Engine : Native / Non-Native
 - Native Graph Storage – optimized for the native graph management.
 - Non-Native Graph Storage – data are stored in non graph based model, but they support a graph query language
 - Relational
 - Object oriented database
 - Wide Column

GRAPH DATABASES



NON-NATIVE : RELATIONAL MODELLING

- Model a graph by means of relational model

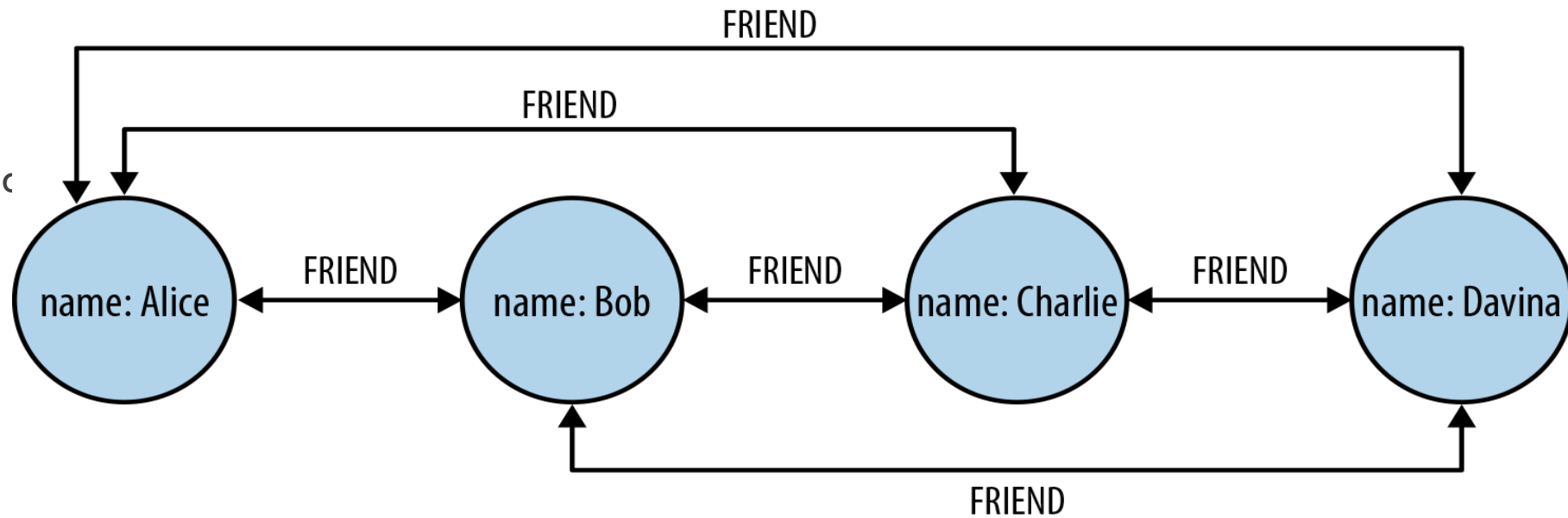


RDBMS

- Usual questions to operational rdbms
- Which products were bought by a customer
- Which customers bought a product
- Which customers who bought a product also bought another?

NATIVE : INDEX FREE ADJACENCY

- In a graph



RDBMS

- Join is based on values

User					
UserID	User	Address	Phone	Email	Alternate
1	Alice	123 Foo St.	12345678	alice@example.org	alice@neo4j.org
2	Bob	456 Bar Ave.		bob@example.org	
...	...	...	...	...	...
99	Zach	99 South St.		zach@example.org	

Order	
OrderID	UserID
1234	1
5678	1
...	...
5588	99

LineItem		
OrderID	ProductID	Quantity
1234	765	2
1234	987	1
...	...	...
5588	765	1

Product		
ProductID	Description	Handling
321	strawberry ice cream	freezer
765	potatoes	
...	...	
987	dried spaghetti	

GRAPH DB –PROCESSING ENGINE

- Index free adjacency – closed nodes are physically stored together
- Very efficient at query time, but very slow at writing time

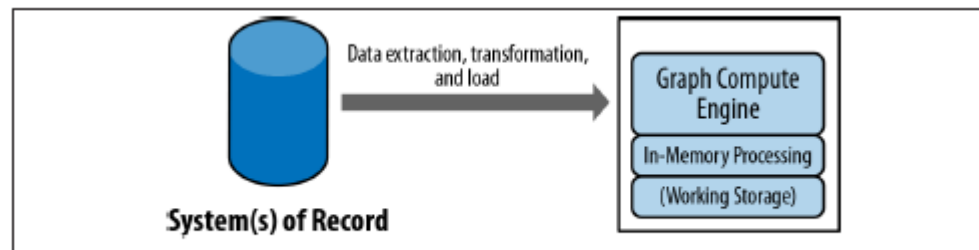
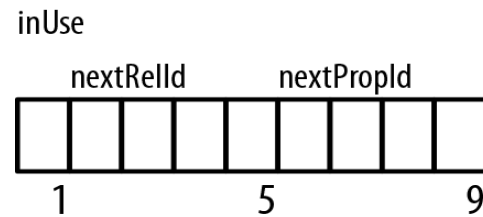


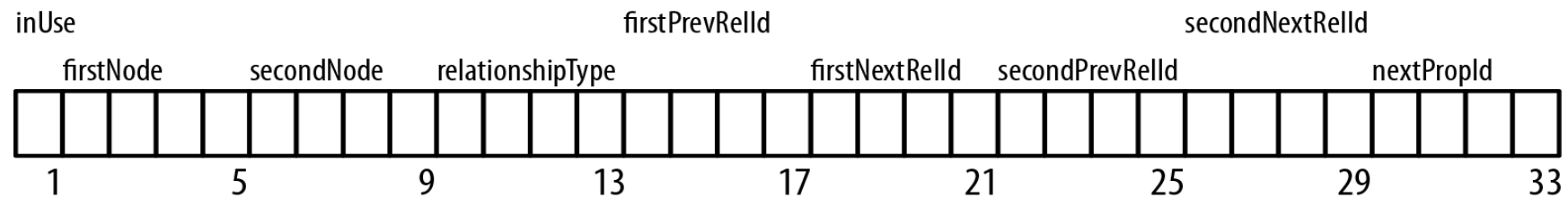
Figure 1-4. A high-level view of a typical graph compute engine deployment

NATIVE GRAPH STORAGE

- Attributes and nodes and referencing to other nodes are stored together

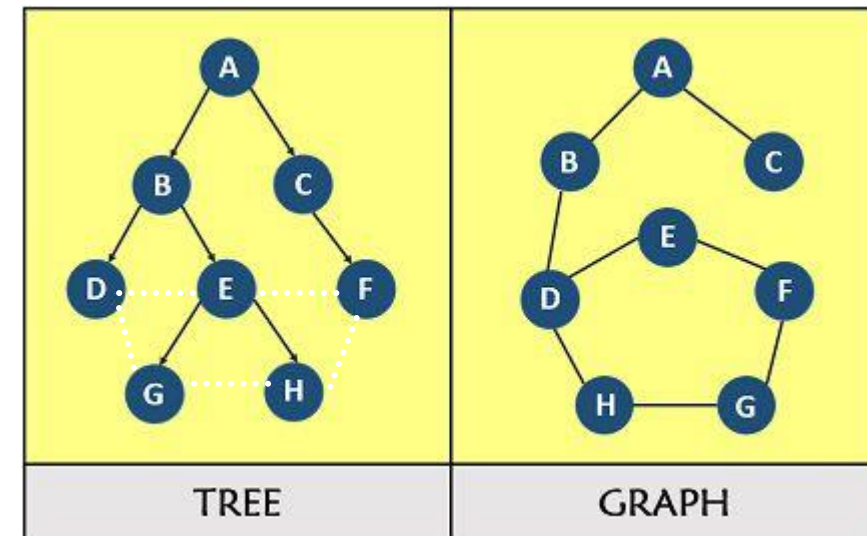


Relationship (33 bytes)

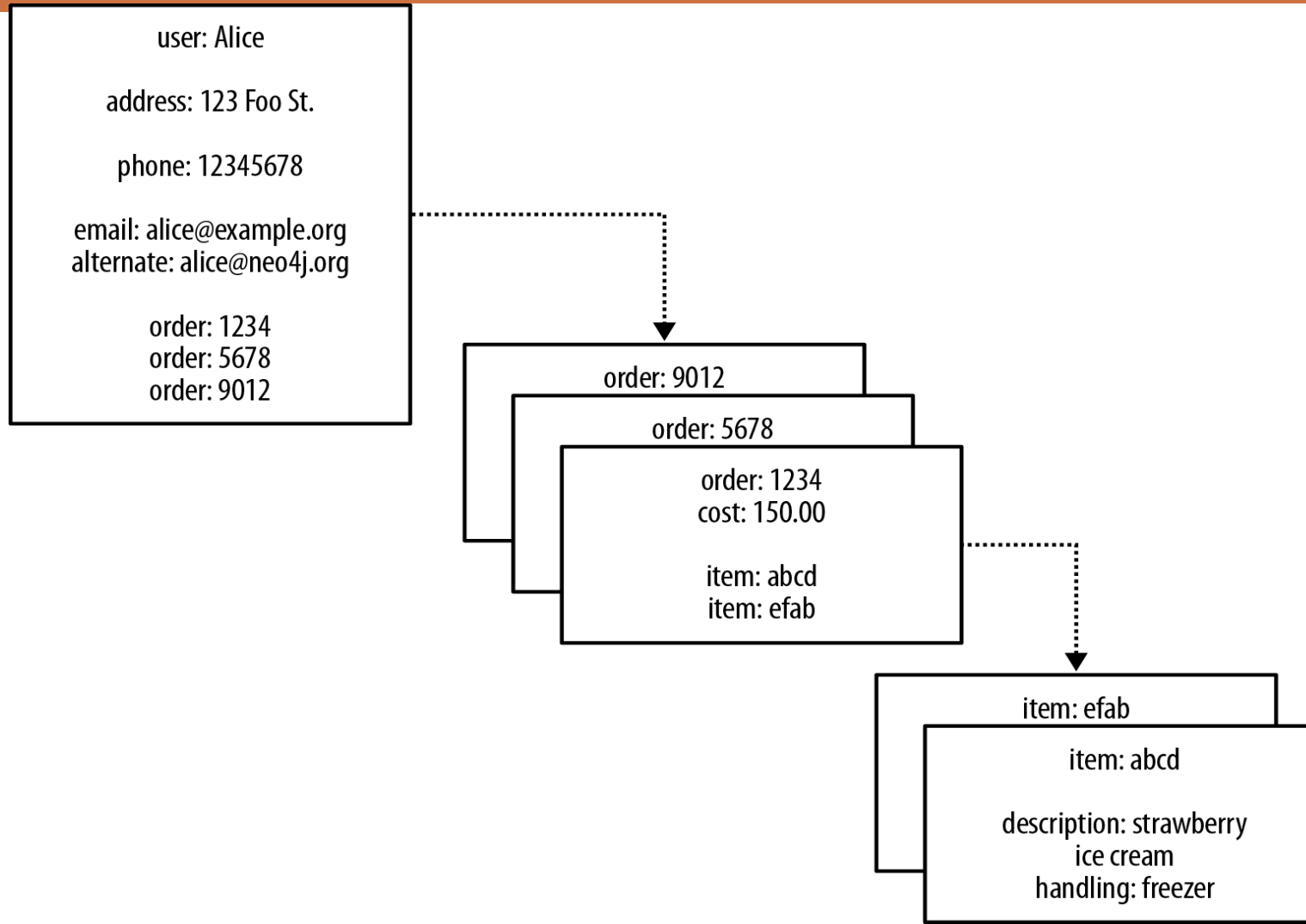


GRAPH AND DOCUMENT BASED

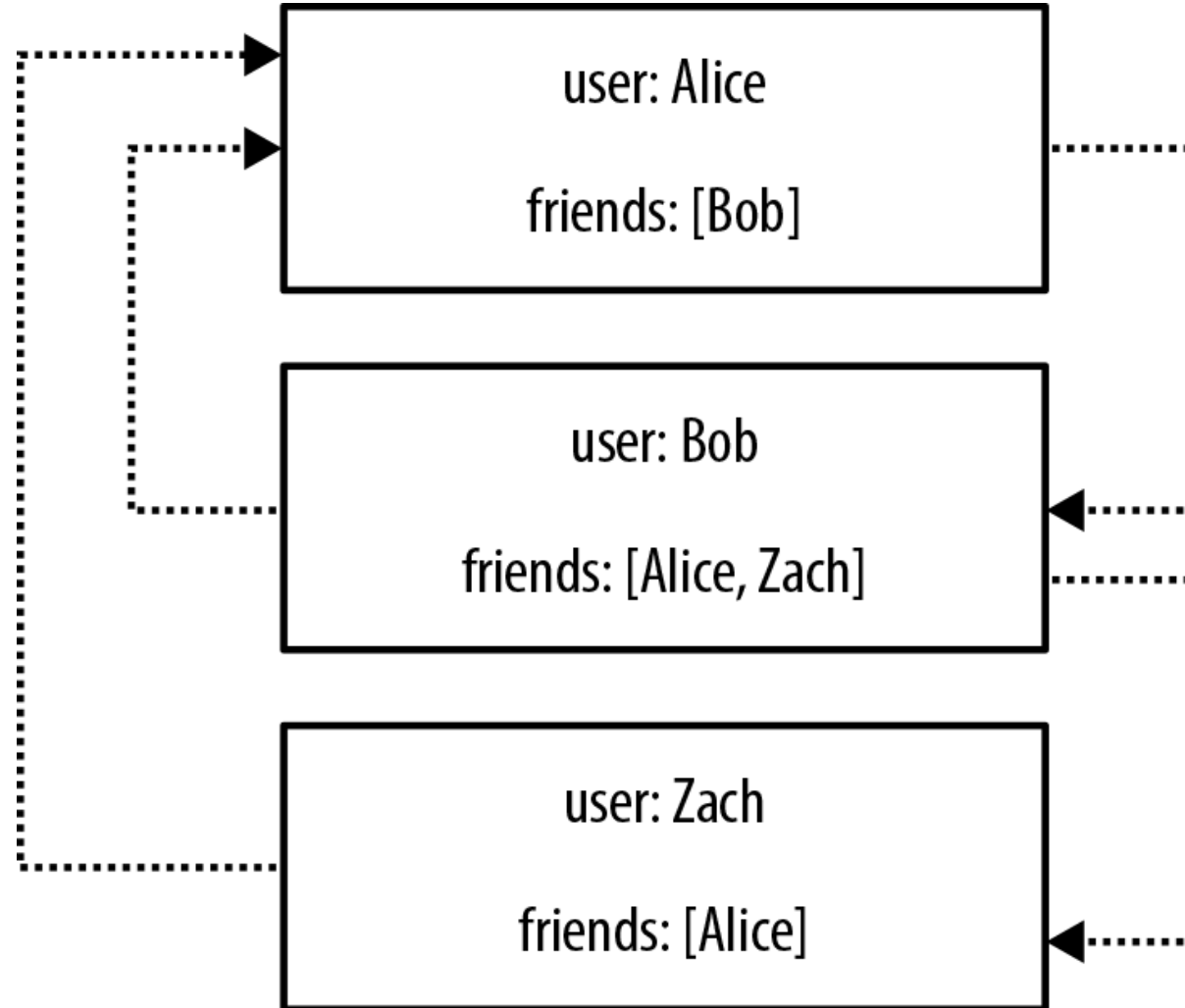
- In document based systems a graph can be modelled by means of identifiers that refer to other documents
- Value based model (the same of relational model)
- Less efficient,
- Modelling is very important



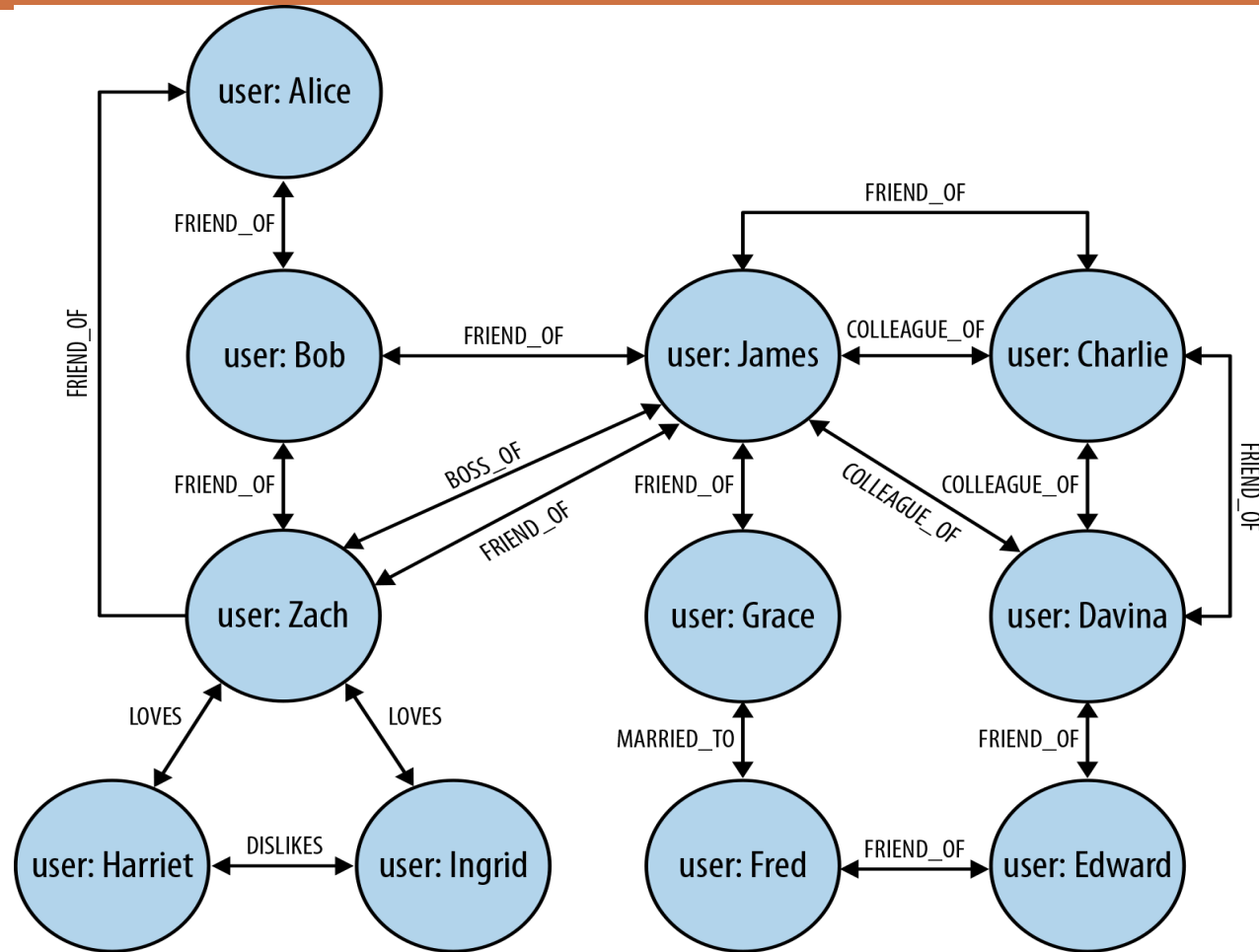
DOCUMENT BASED: EMBEDDING



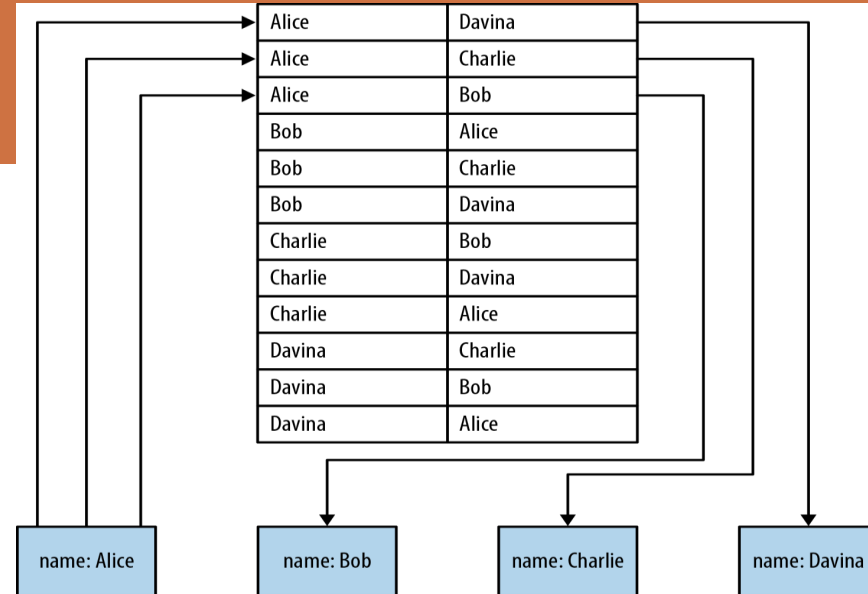
DOCUMENT BASED: REFERENCING



GRAPH DB EMBRACES RELATIONSHIPS



GRAPH DB: EFFICIENCY



- Find friend of a friend at level 5
 - 1,000,000 persons
 - Each person is friend of 50 friends (mean)

Depth	RDBMS execution time (s)	Neo4j execution time (s)	Records returned
2	0.016	0.01	~2500
3	30.267	0.168	~110,000
4	1543.505	1.359	~600,000
5	Unfinished	2.132	~800,000

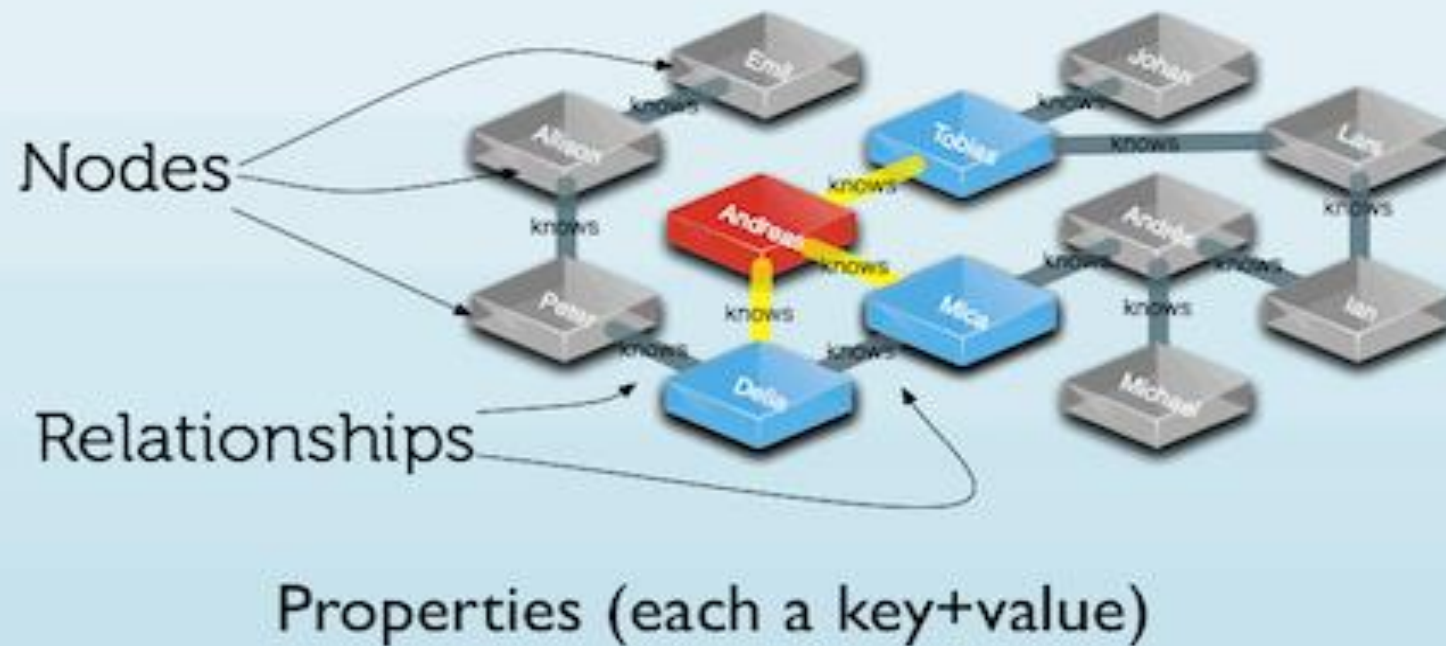


GRAPH MODELLING

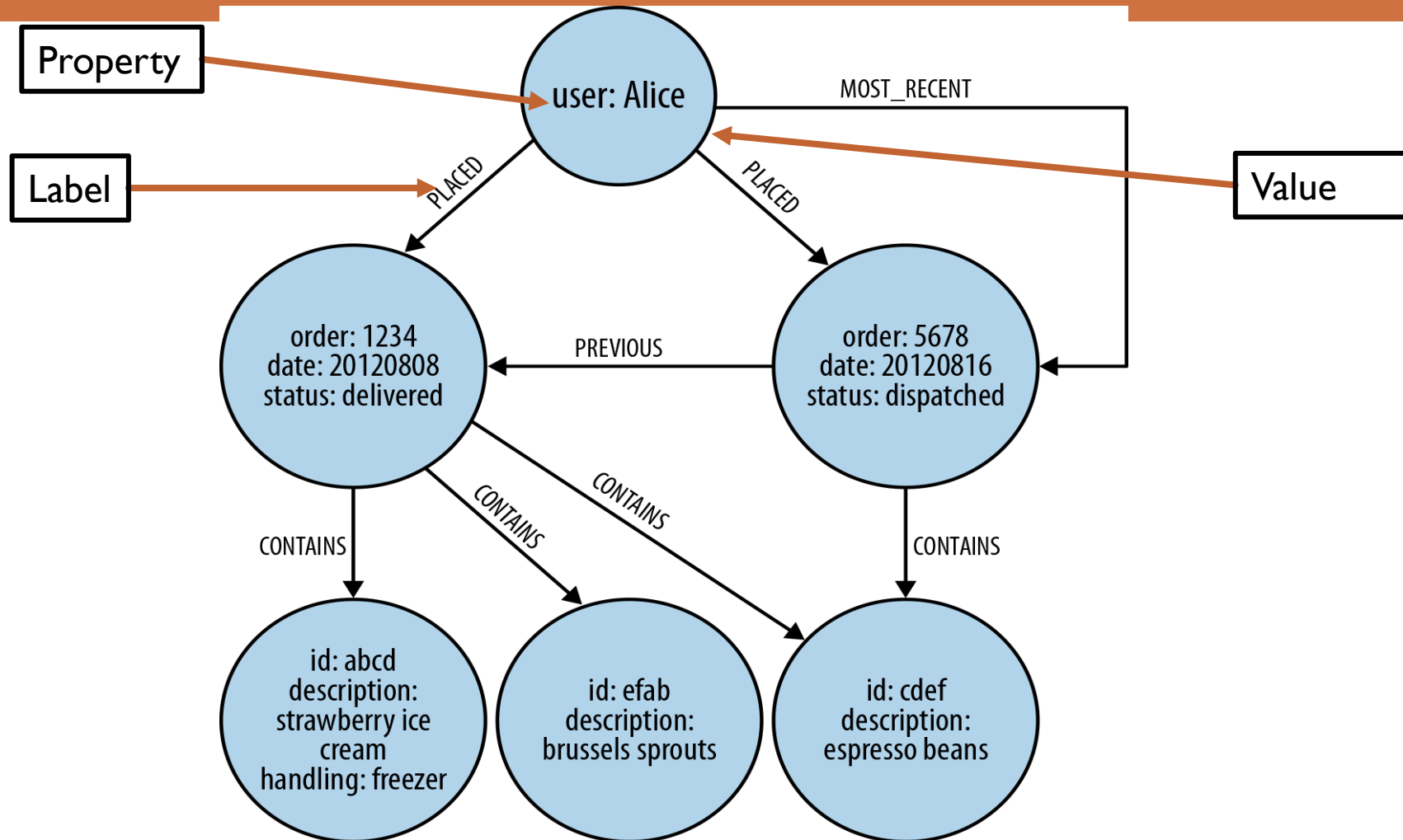


THE PROPERTY GRAPH MODEL

Property Graph



GRAPH DB

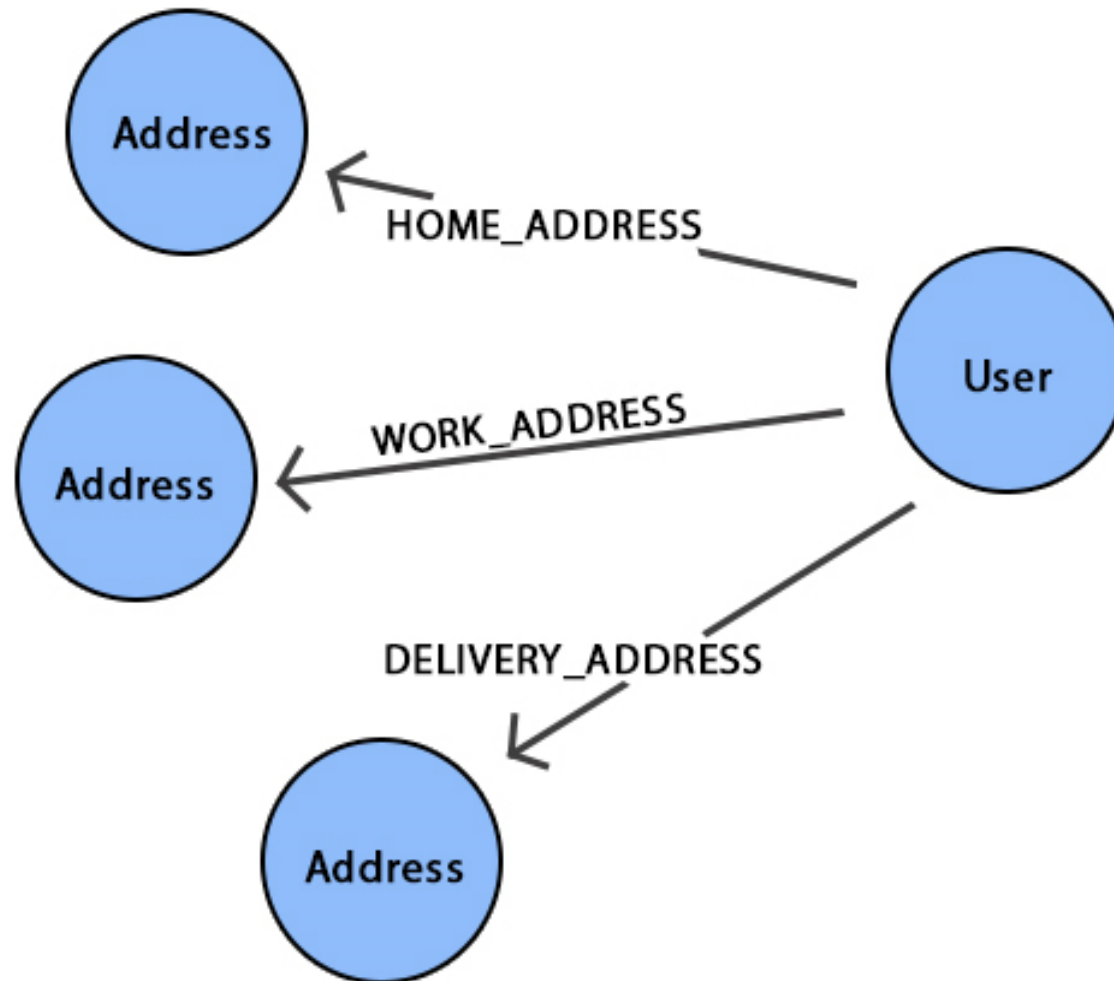


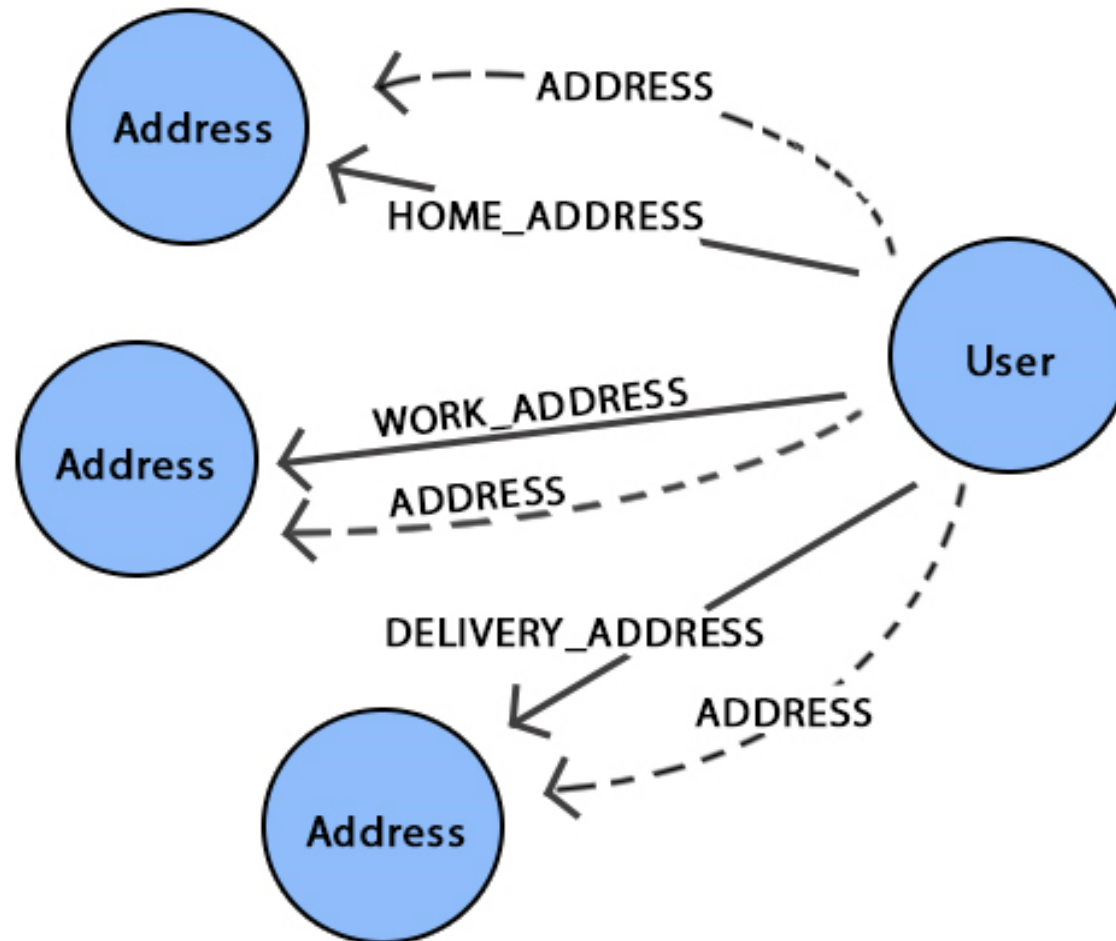
NODES, EDGES, ATTRIBUTES

DELIVERY_ADDRESS

VS

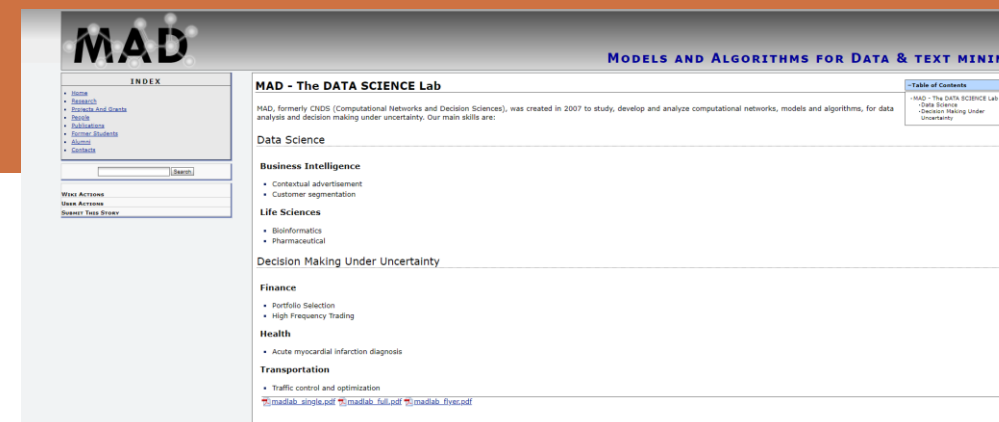
ADDRESS{type : 'delivery'}





AN EXAMPLE

- The problem: In a department there is the need to find the 5 terms representing the top research
- Professor, researchers, post doc and phd students (~100) works in Laboratory (~20)
- Professor, researchers, post doc and phd students publish papers



Towards Improving the Quality of Knowledge Graphs with Data-driven Ontology Patterns and SHACL

Blerina Spahiu, Andrea Maurino, Matteo Palmonari

University of Milano-Bicocca
{blerina.spahiu|andrea.maurino|matteo.palmonari}@unimib.it

Abstract. As Linked Data available on the Web continue to grow, understanding their structure and assessing their quality remains a challenging task making such the bottleneck for their reuse. ABSTAT is an online semantic profiling tool which helps data consumers in better understanding of the data by extracting data-driven ontology patterns and statistics about the data. The SHACL Shapes Constraint Language helps users capturing quality issues in the data by means of constraints. In this paper we propose a methodology to improve the quality of different versions of the data by means of SHACL constraints learned from the semantic profiles produced by ABSTAT.

1 Introduction

Million of facts describing entities such as people, places, events, films, etc., are stored in knowledge repositories. In the Semantic Web, knowledge is encoded into graphs represented using the RDF data model¹. Nodes in these graphs represent *entities* or literals, while arcs represent *relations* between entities and between entities and literals, whose semantics is specified by RDF properties. Entities and literals are usually associated a type, i.e., respectively a class or a datatype. The sets of possible types and properties are organized into *schemas* or *ontologies*, which defines the meaning of the terms used in the knowledge

DATA MODELING

- Design on whiteboard
- Close to human way of works
 - conceptual maps



QUERY LANGUAGES



CYPHER : GRAPH QUERY LANGUAGE

- Pattern-Matching Query Language
- Humane language
- Expressive
- Declarative : Say what you want, non how
- Borrows from well know query languages
- Aggregation, Ordering, Limit
- Update the Graph

CYPHER

➤ syntax

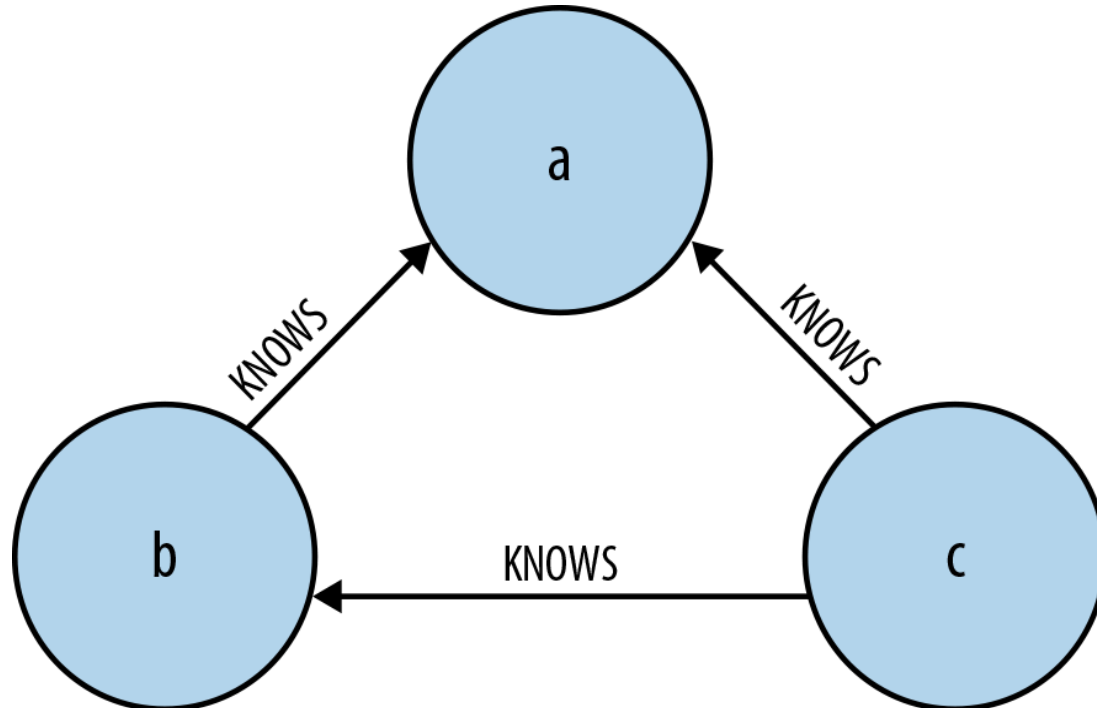
(c)-[:KNOWS]->(b)-[:KNOWS]->(a), (c)-[:KNOWS]->(a)

(c)-[:KNOWS]->(b)-[:KNOWS]->(a)<-[:KNOWS]-(c)

edges

direction

Nodes



CYPHER

MATCH (c:user)

WHERE (c)-[:KNOWS]->(b)-[:KNOWS]->(a), (c)-[:KNOWS]->(a), c.user="Michael"

RETURN a, b

OTHER CYPHER CLAUSES

- **WHERE**
 - Provides criteria for filtering pattern matching results.
- **CREATE and CREATE UNIQUE**
 - Create nodes and relationships
- **DELETE**
 - Removes nodes, relationships and properties
- **SET**
 - Sets property values
- **FOREACH**
 - Performs an updating action for graph element in a list.
- **UNION**
 - Merge results from two or more queries.
- **WITH**
 - Chains subsequent query parts and forward results from one to the next. Similar to piping commands in UNIX.

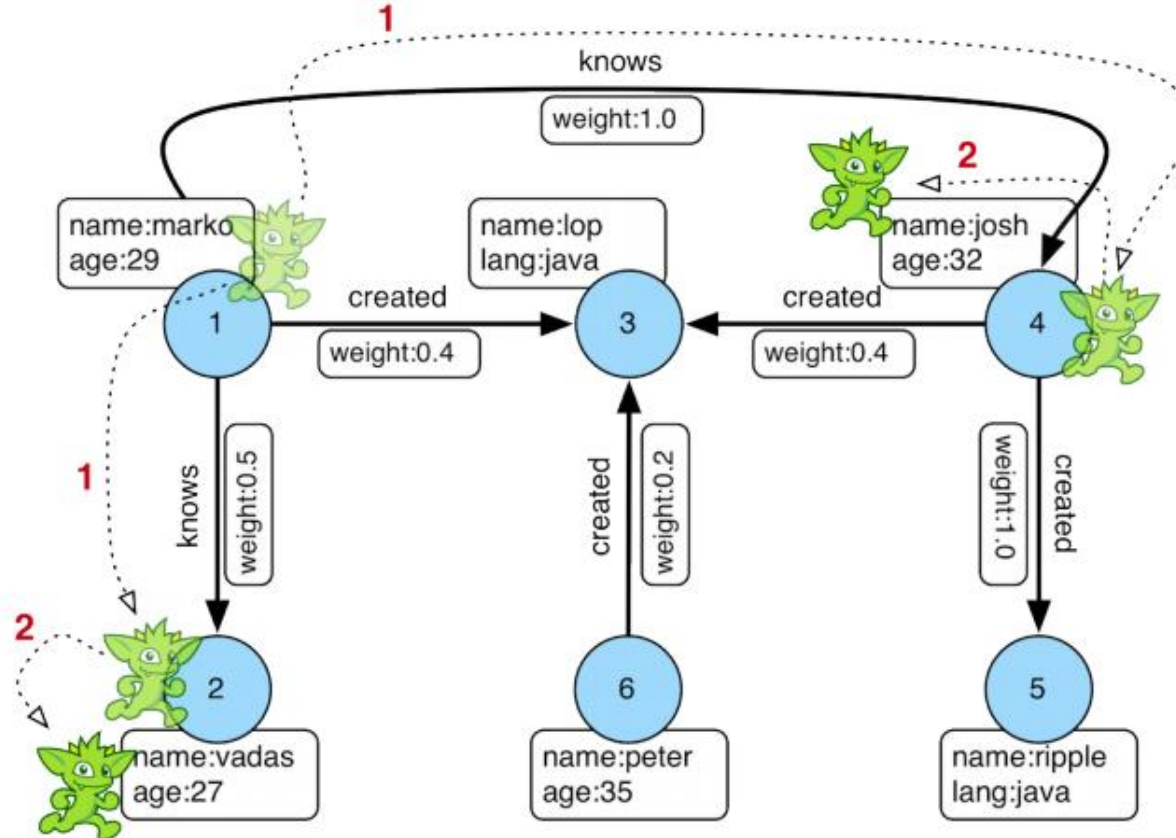
GREMLIN



- Graph Traversal Language
- Part of the Apache TinkerPop framework
- Powerful domain-specific language (DSL) for which embeddings in various programming languages exist
- Expressions specify a concatenation of traversal steps

AN EXAMPLE

- `g.V().has('name','marko').out('knows').values('name')`



SYNTAX

- `g`: for the current graph traversal.
- `V`: all vertices in the graph
- `E`: all edges in the graph
- `has(attribute, value)` filters the vertices down to those with attribute property is equal to name
- `hasLabel(label)`: returns any vertices (nodes) that have the given label.
- `Out(label)`: traverse outgoing labelled edge's from vertices of the previous step.
- `Values(properties)` return the value of properties of nodes produced by previous step
- `Count()`: Count objects of previous step
- `groupCount().by(label)`: count object grouped by label

TRAVERSAL

- *: can optionally take the name of edge labels
- If omitted, all relevant edges will be traversed.
- repeat(out()): repeat the steps (Path= any)
- repeat(out()).times(3): number of repeats (path=3)
- simplePath(): shortest path among two vertices

out *	Outgoing adjacent vertices.
in *	Incoming adjacent vertices.
both *	Both incoming and outgoing adjacent vertices.
outE *	Outgoing incident edges.
inE *	Incoming incident edges.
bothE *	Both outgoing and incoming incident edges.
outV	Outgoing vertex.
inV	Incoming vertex.
otherV	The vertex that was not the vertex we came from.



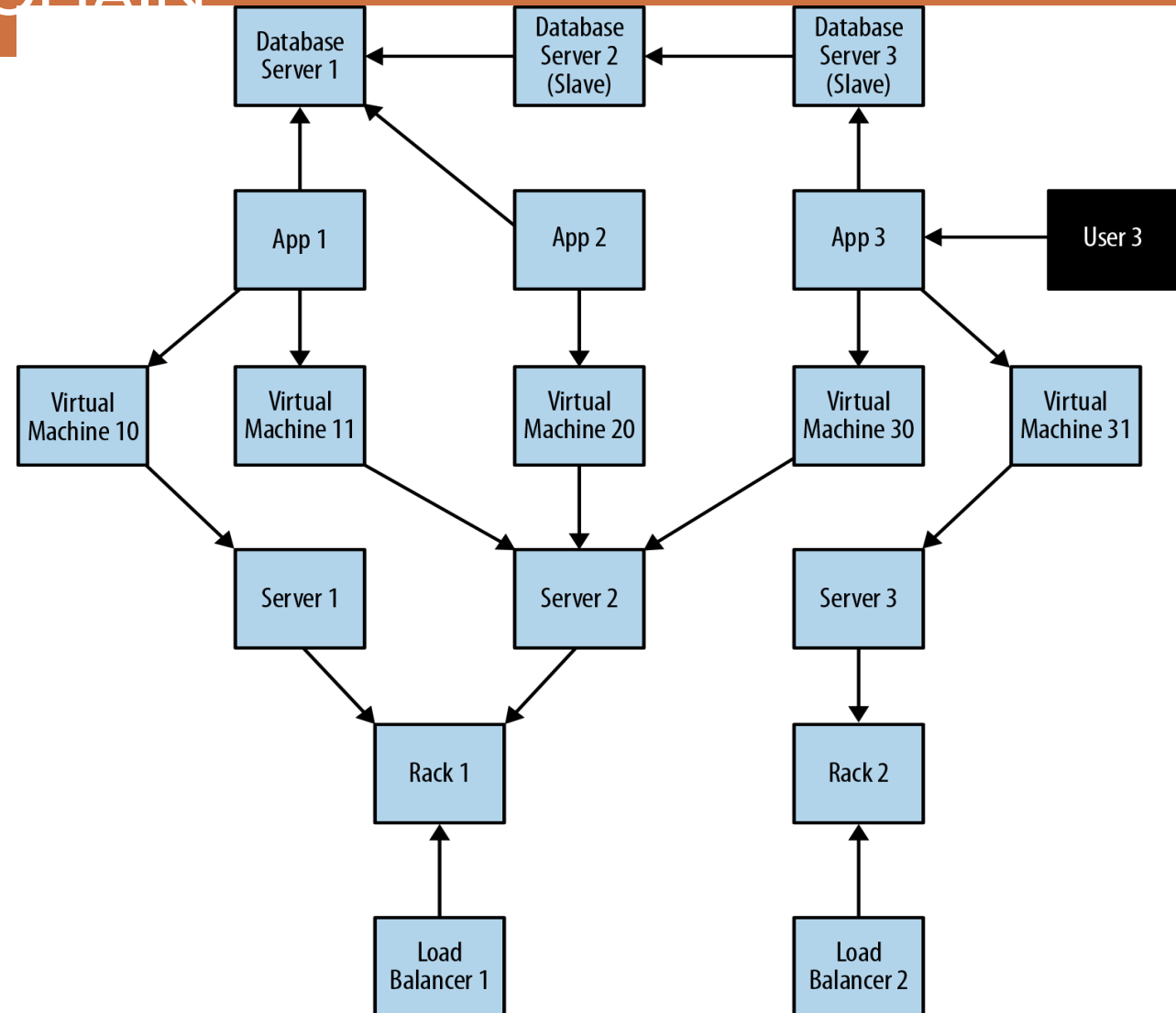
COMPARISON OF RELATIONAL AND GRAPH MODELING



AN EXAMPLE

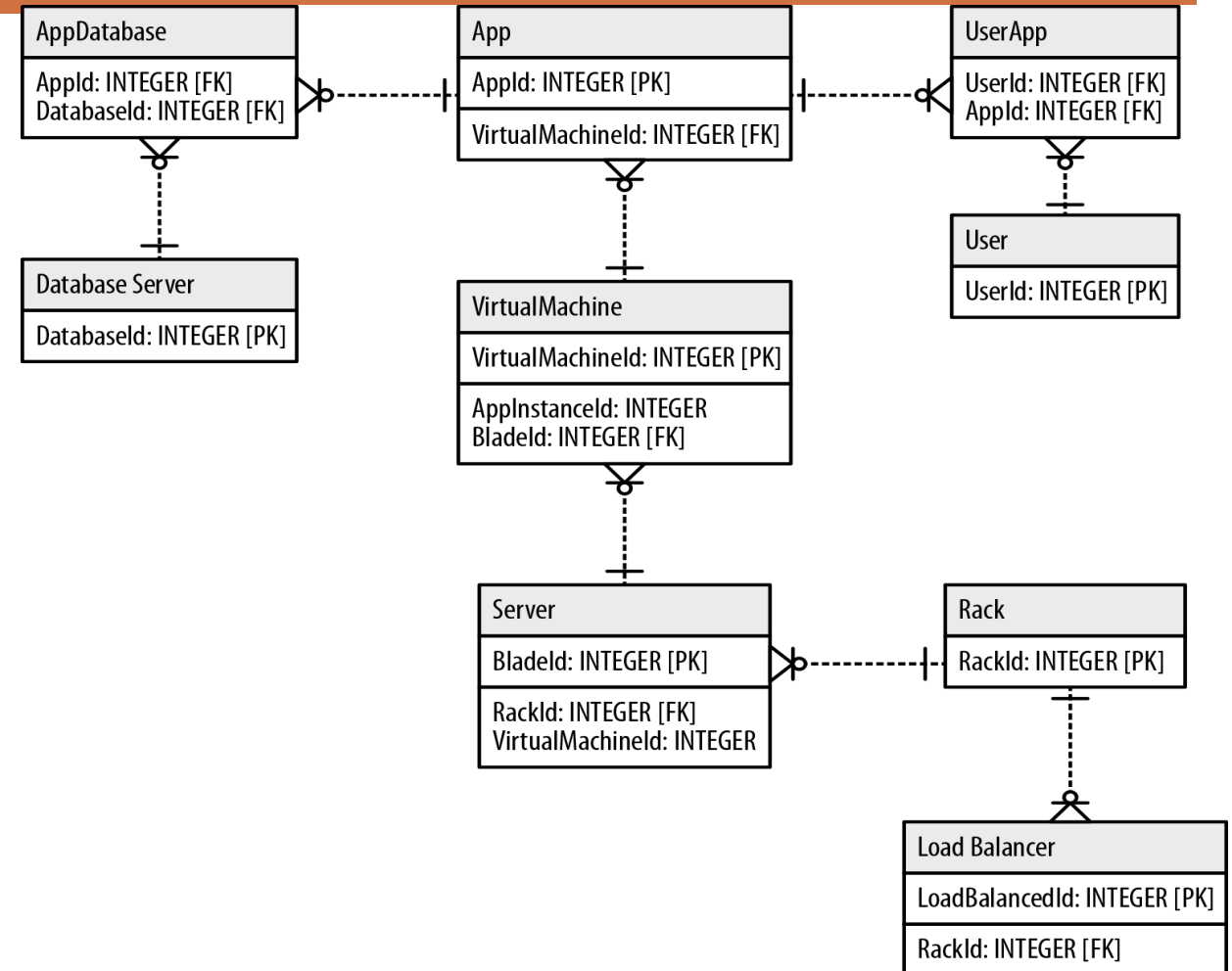
- Define a relational model for managing a server farm
- Users access to application
- Applications run on virtual machines
- Each application uses a database and a secondary one
- Virtual machines are hosted on servers
- Servers are placed in rack structures
- Load balancing system manage racks

SYSTEMS MANAGEMENT DOMAIN

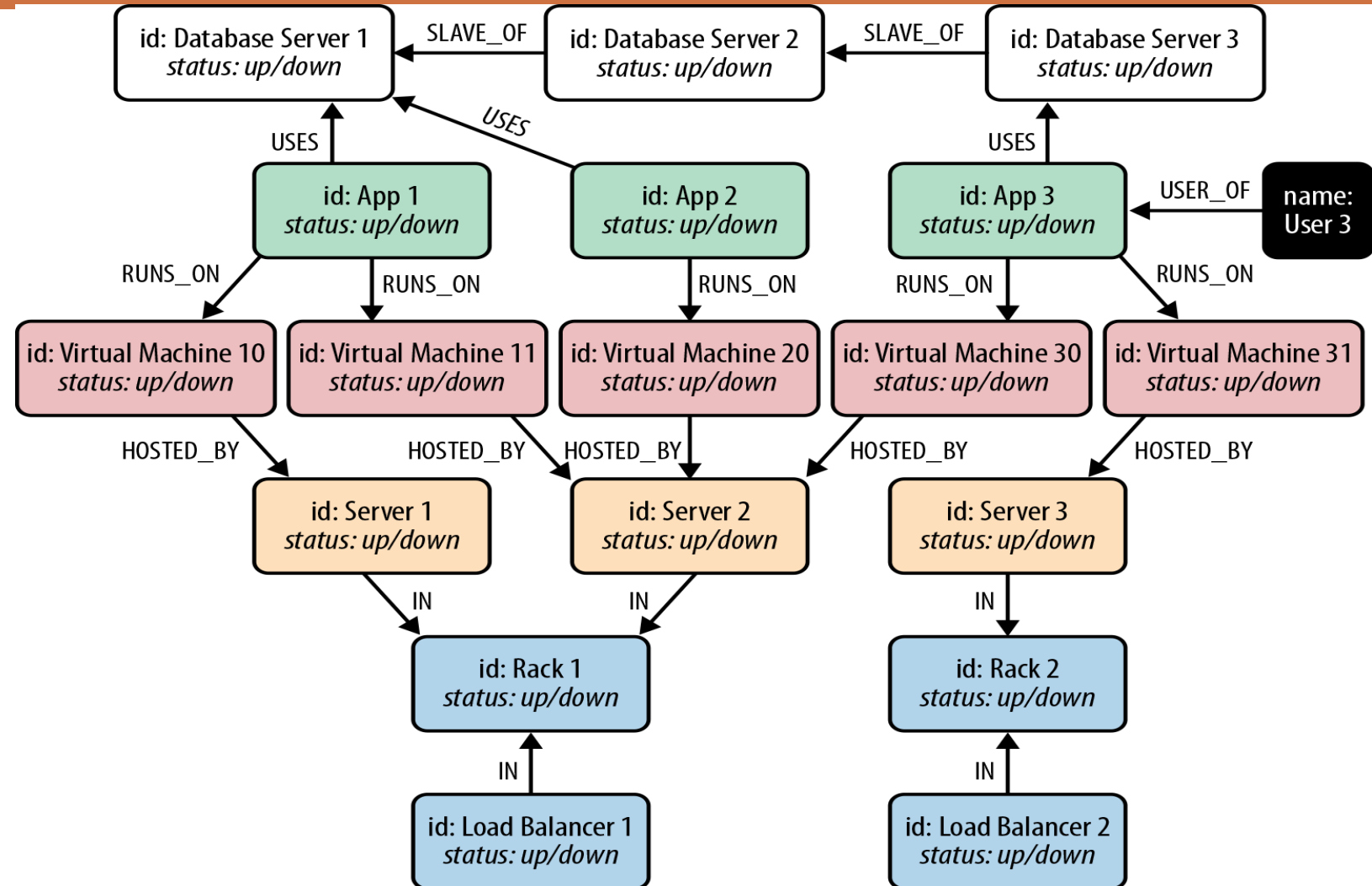


TABLES AND RELATIONSHIPS

- A common attribute is status up and down



GRAPH REPRESENTATION



QUERY TO FIND FAULTY EQUIPMENT

- User 3 notify that the application does not work.
- The app or any other asset related to it (database, virtual machine, server, rack, load balace) can be down
- Find any asset whose status is down for user 3

```
START user=node:users(name = 'User 3')  
MATCH (user)-[*1..5]-(asset)  
WHERE asset.status! = 'down'  
RETURN DISTINCT asset
```

MATCHED PATHS

```
(user)-[:USER_OF]->(app)
(user)-[:USER_OF]->(app)-[:USES]->(database)
(user)-[:USER_OF]->(app)-[:USES]->(database)-[:SLAVE_OF]->(another-database)
(user)-[:USER_OF]->(app)-[:RUNS_ON]->(vm)
(user)-[:USER_OF]->(app)-[:RUNS_ON]->(vm)-[:HOSTED_BY]->(server)
(user)-[:USER_OF]->(app)-[:RUNS_ON]->(vm)-[:HOSTED_BY]->(server)-[:IN]->(rack)
(user)-[:USER_OF]->(app)-[:RUNS_ON]->(vm)-[:HOSTED_BY]->(server)-[:IN]->(rack)
    <-[:IN]-(load-balancer)
```